Sesión 8

Modelos Seq2Seq y Atención

Carlos César Sánchez Coronel

NLP

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Problemas de secuencia a secuencia

- Muchas tareas de NLP requieren transformar una secuencia de entrada en una secuencia de salida de longitud potencialmente diferente.
- Ejemplos:
  - **Traducción automática**: inglés $\rightarrow$ francés.
  - **Summarization**: artículo largo $\rightarrow$ resumen corto.
  - **Diálogo**: pregunta del usuario $\rightarrow$ respuesta del sistema.
  - **Generación de preguntas**: texto $\rightarrow$ pregunta relevante.

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Visión general

Encoder

- RNN (LSTM/GRU) que procesa la entrada $x_1, \ldots, x_T$.
- Produce estados ocultos $h_1, \ldots, h_T$.
- El último estado h_T (o combinación) se usa como **vector de contexto** c.

Decoder

- Otra RNN que genera la salida $y_1, \ldots, y_{T'}$.
- Estado inicial $s_0 = c$.
- En cada paso, predice y_t basado en s_{t-1} y y_{t-1}.

Limitación del contexto fijo

Cuello de botella

El vector de contexto c debe comprimir toda la información de la entrada en una representación de dimensión fija. Para secuencias largas, la información se diluye y se pierden detalles.

- Ejemplo: "El gato que estaba durmiendo en el sofá gris persiguió al ratón". El modelo sin atención puede confundir quién persiguió a quién.

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Intuición de la atención

- Permite al decodificador, en cada paso, “mirar” directamente a todos los estados del encoder y decidir qué partes son más relevantes.
- Resuelve el cuello de botella del contexto fijo.
- Mejora el rendimiento en secuencias largas y proporciona interpretabilidad (pesos de atención).

Atención de Bahdanau (Additive Attention)

Cálculo

$$e_{t,i} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

donde $\mathbf{s}_{t-1}$ es el estado oculto anterior del decodificador, $\mathbf{h}_i$ son los estados del encoder.

Atención de Luong (Multiplicative Attention)

- Usa el estado actual $\mathbf{s}_t$ (después de actualizar con la entrada).
- Variantes de puntuación:
  - **Producto punto**: $\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^\top \mathbf{h}_i$
  - **General**: $\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^\top \mathbf{W}_a \mathbf{h}_i$
  - **Concat**: similar a Bahdanau pero con $\mathbf{s}_t$
- Más eficiente computacionalmente, especialmente producto punto.

Atención global vs local

Atención global

- Considera todos los estados del encoder.
- Más costosa, pero captura toda la información.

Atención local

- Predice una posición alineada y usa una ventana alrededor.
- Más eficiente, puede perder información lejana.

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Encoder LSTM

```
import torch
import torch.nn as nn

class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=1, dropout=0.5):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
                            batch_first=True, dropout=dropout)
    def forward(self, x):
        embedded = self.embedding(x)          # (batch, seq_len, embed_dim)
        outputs, (hidden, cell) = self.lstm(embedded)
        return outputs, hidden, cell
```

Decoder con Atención (Luong)

```python
class DecoderWithAttention(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=1, dropout=0.5):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers, batch_first=True)
        self.attn_proj = nn.Linear(hidden_dim * 2, hidden_dim)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, y_prev, hidden, cell, encoder_outputs):
        embedded = self.embedding(y_prev)                          # (batch, 1, embed_dim)
        lstm_out, (hidden, cell) = self.lstm(embedded, (hidden, cell))
        query = lstm_out.squeeze(1)                                # (batch, hidden_dim)
        scores = torch.bmm(encoder_outputs, query.unsqueeze(2)).squeeze(2)
        attn_weights = torch.softmax(scores, dim=1)                # (batch, seq_len)
        context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs).squeeze(1)
        combined = torch.tanh(self.attn_proj(torch.cat((query, context), dim=1)))
        output = self.fc_out(combined)                              # (batch, vocab_size)
        return output, hidden, cell, attn_weights
```

Modelo Seq2Seq completo

```
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device

    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size, trg_len = trg.shape
        trg_vocab_size = self.decoder.fc_out.out_features
        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size).to(self.device)
        encoder_outputs, hidden, cell = self.encoder(src)
        decoder_input = torch.ones(batch_size, 1).long().to(self.device)  # <SOS>
        for t in range(trg_len):
            output, hidden, cell, attn = self.decoder(decoder_input, hidden, cell,
                encoder_outputs)
            outputs[:, t, :] = output
            # Teacher forcing
            if torch.rand(1).item() < teacher_forcing_ratio:
                decoder_input = trg[:, t].unsqueeze(1)
            else:
                decoder_input = output.argmax(1).unsqueeze(1)
        return outputs
```

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

BLEU (Bilingual Evaluation Understudy)

- Métrica estándar para evaluar traducción automática.
- Compara la traducción generada con una o más referencias humanas.
- Se basa en la precisión de n-gramas (normalmente $n = 1, 2, 3, 4$) con una penalización por longitud (brevity penalty).

Cálculo de BLEU

Fórmula

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{4} w_n \log p_n\right)$$

donde $w_n = 1/4$, p_n es la precisión de n-gramas, y BP es la penalización por longitud:

$$\text{BP} = \begin{cases} 1 & \text{si } c > r \\ e^{(1-r/c)} & \text{si } c \leq r \end{cases}$$

c = longitud del candidato, r = longitud de la referencia.

Ejemplo numérico

Referencia: "the cat sits on the mat"

Candidato: "the cat on the mat"

- Unigramas: $5/5 = 1.0$
- Bigramas: $3/4 = 0.75$
- BP: $c = 5$, $r = 6 \Rightarrow BP = e^{(1-6/5)} = e^{-0{,}2} = 0{,}8187$
- BLEU (n=1,2) $= 0{,}8187 \times \exp(0{,}5 \cdot \log 1 + 0{,}5 \cdot \log 0{,}75) \approx 0{,}709$

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Traducción automática

- Google Translate y DeepL utilizaron arquitecturas Seq2Seq con atención antes de los Transformers.
- La atención permite manejar frases largas y mejorar la calidad.

Summarization abstractivo

- Modelos como BART y T5 generan resúmenes a partir de documentos largos.
- La atención permite enfocarse en las partes relevantes del texto original.

Diálogo y chatbots

- Sistemas de diálogo (atención al cliente) usan Seq2Seq con atención para mantener el contexto de la conversación.
- La atención ayuda a recordar información de turnos anteriores.

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Preguntas típicas

- ¿Cómo funciona el mecanismo de atención?
- Diferencia entre atención de Bahdanau y Luong.
- ¿Qué es teacher forcing y por qué se usa?
- Explica la métrica BLEU.
- ¿Por qué el contexto fijo de Seq2Seq básico es problemático?

Preguntas avanzadas

- ¿Qué es beam search y cómo mejora la generación?
- ¿Cómo manejarías secuencias de longitud variable en un lote?
- ¿Qué es el problema de exposure bias?
- ¿Qué es copy mechanism y cuándo es útil?
- ¿Qué ventajas tienen los Transformers sobre Seq2Seq con atención?

Contenido

1. Introducción a Problemas Seq2Seq
2. Arquitectura Encoder-Decoder
3. Mecanismo de Atención
4. Implementación en PyTorch
5. Métrica BLEU
6. Aplicaciones en la Industria
7. Preparación para Entrevistas
8. Resumen

Lo que hemos aprendido

Conceptos clave

- **Seq2Seq**: arquitectura encoder-decoder para transformar secuencias.
- **Vector de contexto fijo**: cuello de botella para secuencias largas.
- **Atención**: permite al decoder enfocarse en partes relevantes de la entrada.
- **Bahdanau (additive)** vs **Luong (multiplicative)**: diferentes formulaciones.
- **Atención global vs local**: trade-off precisión/eficiencia.
- **BLEU**: métrica estándar para traducción basada en n-gramas.
- **Implementación**: encoder, decoder con atención, y entrenamiento en PyTorch.
- **Aplicaciones**: traducción, summarization, diálogo.
- **Técnicas avanzadas**: beam search, copy mechanism.

¡Gracias!